

Proyecto Integrador Documentación

Objetivos

- Construir un servicio de Orders con dominio, casos de uso, puertos y adaptadores
- Exponer API FastAPI segura y documentada.
- Asegurar calidad (pruebas, lint, tipado) y delivery (Docker, CI/CD)

Arquitectura Hexagonal

El presente proyecto fue construido en base a la arquitectura hexagonal, donde el sistema está dividido en tres capas principales con reglas estrictas de dependencia (las dependencias siempre apuntan hacia el interior):

1. **Capa de Dominio:** Contiene las Entidades (Order, OrderItem). En esta capa residen las reglas de negocio estrictas, como:
 - Una orden no puede crearse sin artículos (items).
 - El correo electrónico del cliente debe tener un formato válido.
 - El cálculo del total se realiza dinámicamente (total_amount).
2. **Capa de Aplicación (Orquestador y Contratos):** Contiene:
 - Puertos: Se definen interfaces abstractas (OrderRepository) donde se dictan qué operaciones se pueden hacer (crear, listar, obtener por ID, actualizar, eliminar) sin especificar cómo se harán.
 - Casos de Uso: La clase OrderUseCase coordina las operaciones. Recibe los datos crudos, valida e instancia las entidades de Dominio, y luego utiliza el Puerto para persistir los cambios.
3. **Capa de Infraestructura (Adaptadores):** Donde el software interactúa con Fast API y SQLAlchemy:
 - **Adaptador de Entrada** (FastAPI): Expone los endpoints HTTP. Valida los JSON de entrada y salida mediante Pydantic (Esquemas / DTOs) y maneja la seguridad mediante tokens JWT.
 - **Adaptador de Salida** (SQLAlchemy + Alembic): Implementa el repositorio real (SQLAlchemyOrderRepository) traduciendo las entidades puras del Dominio en modelos de base de datos relacional (ORM) para guardarlos en SQLite.

Pruebas

El proyecto cuenta con 3 tipos de pruebas:

1. **Unitarias:** Prueban el Dominio y la Aplicación, usando Mocks (MagicMock) para aislarlo de la base de datos.
2. **De contrato:** Verifican que el Adaptador de Salida (SQLAlchemy) realmente se conecte a una base de datos en memoria y ejecute las sentencias SQL de manera correcta.
3. **E2E:** Pruebas que simulan peticiones web reales mediante el TestClient de FastAPI, validando desde los candados de seguridad JWT hasta el almacenamiento físico.

Docker

Se empaqueta la aplicación usando Dockerfile y Docker-compose permitiendo ejecutar y levantar la aplicación de manera rápida, con solo hacer uso del comando `docker-compose up -d --build`, dicho contenedor contiene las siguientes características:

- Multistage: Consiste de dos etapas, donde la primera, el builder instala las herramientas de desarrollo, poetry y el archivo binario .whl; la segunda, el runner, copia únicamente el paquete, reduciendo el tamaño.
- Hardening: Se crea un grupo y usuario específico sin privilegios de administrador.
- Entrypoint encadenado: Al iniciar el contenedor, se ejecutab los scripts de migración Alembic antes de inicar Uvicorn y FastAPI.

Integración y Entrega Continua (CI/CD)

Mediane GitHub Actions (por medio del archivo `ci.yml`) el proyecto está conectado a un pipeline automatizado, pues realiza las siguientes tareas tras haber realizado un `git push` (tanto en rama main como en develop), así como también al realizar un `git merge`:

1. Auditoría de Seguridad: Mediante `pip-audit` escanea las dependencias en busca de vulnerabilidades.
2. Linter y formato: En esta tarea se ejecutan `ruff check` y `ruff format`.
3. Análisis de Tipos estáticos: Se ejecuta `mypy` asegurando la consistencia en el código.
4. Testing: Corre los 3 tipos de pruebas (Unitarias, de Contrato y E2E) con una cobertura del código.
5. Imagen Docker: Si todas las anteriores tareas pasan con éxito, se construye la imagen y verifica que funcione de manera correcta.